Master 2027 Conference – Track 1 (Artificial Intelligence)

Two Research Proposals in Penetration Testing and AI: Can Small Local Models Be Relied On Without “Making Things Up”?

Two proposals for a short paper (4 pages, IEEE format) | Target conference: MASTER 2027 – Track 1 (AI) | October 2026

1. Overview

Both ideas share a single question: can a small language model running locally on a professional’s machine be trusted with a penetration-testing task, and how often does it “hallucinate” information that does not exist? We measure this by comparing its output against a correct answer known in advance. Running locally is deliberate: real client data is sensitive and must not be sent to an external service.

	Idea 1	Idea 2
Topic	Converting Nmap results into a list of vulnerabilities (CVEs)	Extracting hidden endpoints and secrets from minified and obfuscated JavaScript code
Phase	Network and service discovery	Web application discovery

2. Key Terms (simplified)

Term	Meaning
CVE	An official identifier for a known security vulnerability in a specific piece of software, e.g. CVE-2021-41773 (a vulnerability in Apache 2.4.49).
NVD	The (U.S. government) National Vulnerability Database, which holds all CVEs and the versions they affect.
Nmap	A scanning tool that detects the services running on a machine or server, and their versions, but does not report vulnerabilities.
CPE	A standard name for a piece of software together with its version, used to link it to its vulnerabilities in NVD.
Endpoint	An address on a website that accepts requests, such as /api/users. Testers look for these because they are potential attack surfaces.
Minified / Obfuscated	Code whose whitespace has been removed and names shortened (minified), or which has been deliberately made hard to understand (obfuscated).
Regex	A simple text-search pattern that catches anything resembling a given shape (such as /api/...).
LLM	A large language model that understands text and generates responses to it (such as GPT).
Ollama	A tool for running small language models on your own machine without sending data to the internet.
Hallucination	When a model “invents” incorrect information while sounding confident about it.
Ground Truth	The correct answer known in advance — the “answer sheet” against which we grade the model’s output.

Idea 1

Can Small Local LLMs Map Nmap Results to CVEs Without Hallucinating?

An Empirical Evaluation

The idea in brief

An experimental study measuring the ability of small local language models to convert Nmap scan results into a list of known vulnerabilities (CVEs), with a focus on measuring **hallucination**: the model citing a CVE number that does not exist at all, or one that exists but does not apply to the scanned service. We do not just ask "is the model accurate?" but rather **"can what it says be trusted?"**

The project story in plain words

Step 1	**Step 2**	**Step 3**
Scan the server with Nmap. It shows, for example, Apache 2.4.49.	Manually search NVD for vulnerabilities in this version.	Write the vulnerabilities into the client report.

The idea: hand Step 2 to a small model running on the tester's own machine: we give it the Nmap result and it returns a list of likely vulnerabilities.

The problem we will study: the model may go wrong in one of four ways:

- **Correct:** cites a CVE that really exists for this version.
- **Real but wrong:** cites a CVE that exists in NVD but belongs to a different program or version.
- **Fabricated:** cites a number that looks real but does not exist anywhere.
- **Missing:** omits real vulnerabilities that do exist.

This is dangerous in practice: a fabricated CVE inside a client report destroys the report's credibility.

The role of AI

The model acts as a "candidate assistant": it reads the Nmap output, identifies the product and version, and proposes CVEs with a confidence level; it is also allowed to answer "not sure." We do not assume its answer is correct — we verify it automatically against NVD. We will test three prompt styles: (a) a direct question, (b) the same question plus "say 'not sure' and do not invent numbers," and (c) giving the model a shortlist of CVEs from NVD to choose from only.

Research questions

1. What proportion of the CVEs in the models' output are correct, real-but-wrong, and fabricated?
2. Do the "say 'not sure'" instruction or providing a shortlist reduce hallucination?
3. Does the answer stay stable when the same question is repeated?
4. Does the model add value compared with a non-AI baseline (CPE from Nmap + direct NVD lookup)?

Methodology and metrics

1. **Environment:** run 20–30 services with known versions (Apache, Nginx, PHP, MySQL, OpenSSH, vsftpd, and others) inside Docker on my own machine, including recent versions free of known vulnerabilities to test whether the model "invents" vulnerabilities for a healthy service.
2. **Correct answer:** a table of the real CVEs for each version taken directly from NVD (not from my memory or the model's), with a manual review of a sample.
3. **Execution:** `nmap -sV` on each service, then a Python script sends the results to 2–3 models via Ollama using the three prompt styles, repeating each question 3–5 times.

4. **Automated grading:** check every CVE the model cites against NVD and classify it, comparing with the baseline.

Metrics: Precision / Recall, hallucination rate (fabricated ÷ everything the model cited), the "real but wrong" rate, the abstention rate ("not sure"), answer stability across repetitions, and response time. Expected experiment size: roughly 500–800 calls.

Prior work and expected contribution

- A master's thesis (Cyber Defense, 2026) evaluated 36 models on converting Nmap results to CVE/CPE, but on **Internet of Things (IoT) devices**.
- The study "Using LLMs for Security Advisory Investigations: How Far Are We?" (arXiv:2506.13161) tested ChatGPT on 100 real CVE numbers and 100 fabricated ones, and showed it is weak at telling them apart.

Contribution: focus on web services and servers rather than IoT; measure **hallucination and stability** as independent indicators instead of accuracy alone; use **small local models** rather than ChatGPT; and include an explicit comparison with a **non-AI baseline**.

Idea 2

Can Small Local LLMs Extract Endpoints and Secrets from Minified and Obfuscated JavaScript?

An Empirical Evaluation

The idea in brief

During discovery, the tester opens a website's JavaScript files to look for API endpoints, admin panels not visible to users, and secrets buried in the code. Ready-made tools mostly rely on Regex or code-structure analysis. We propose to measure: **when code is minified and obfuscated to varying degrees, does a small local model outperform Regex? And does it "invent" endpoints and secrets that are not there?**

The project story in plain words

Step 1	Step 2	Step 3
The site loads a JavaScript file containing its API endpoints.	The tester extracts the endpoints and secrets from the file (often minified or obfuscated).	They test those endpoints for vulnerabilities.

The idea: assign Step 2 to a small local model and compare its performance with traditional methods at each level of complexity.

The problem we will study: the model or the Regex may go wrong in three ways:

- **Correct:** extracts an endpoint or secret that is actually in the code.
- **Missing:** misses endpoints or secrets that exist (expected under heavy obfuscation).
- **Fabricated:** cites an endpoint or secret that is not in the original code.

Both approaches may fail under heavy obfuscation; this is an acceptable scientific result, reported as a limit of each method.

The role of AI

The model reads the file in chunks (because large files exceed its capacity) and returns a JSON list of endpoints and secrets, **together with the verbatim snippet of code from which it inferred each item**. We treat its answer as a suggestion that is verified automatically by text comparison, and compare it with a non-AI baseline (Regex or a ready-made tool such as Harpy).

Research questions

1. How do recall and precision change as the level of obfuscation increases? Where does the model beat Regex, and where does it fail?
2. What proportion of the endpoints and secrets in the model's output are fabricated?
3. Is the snippet the model cites as evidence actually present in the file? Is the answer stable across repetitions?
4. What is the effect of splitting large files on extraction quality?

Methodology and metrics

1. **Preparation:** 10–15 small JavaScript applications (from my own projects or parts of the open-source OWASP Juice Shop) containing known endpoints, hidden endpoints, and fake secrets that I plant myself, so the correct answer is known from the original code.
2. **Five versions of each file:** original, minified, identifier renaming, String Array encoding, and encoding combined with Control Flow Flattening, using the JavaScript-Obfuscator tool.
3. **Execution:** the baseline (Regex), then 2–3 models via Ollama with file chunking, repeating each experiment 3–5 times.
4. **Automated grading:** compare every extracted item with the correct answer and match the quoted snippet against the file text.

Metrics: Precision / Recall per obfuscation level, hallucination rate (items not present in the original code), correctness of the textual evidence, answer stability, and response time.

Prior work and expected contribution

- Many ready-made tools extract endpoints from JavaScript (such as Harpy, which combines Regex and AST analysis). They are not evaluation studies, and we will use them as the baseline.
- Research on LLM deobfuscation: the JsDeObsBench benchmark (arXiv:2506.20170) tested models on 1,298 programs to which seven obfuscation transformations were applied using JavaScript-Obfuscator, and showed that many deobfuscated programs fail at execution (37.4% on average). There is also a deobfuscation tool presented at NDSS 2026.

> **Contribution:** those works measure "is the deobfuscated code correct and readable?", whereas we measure the tester's **actual task**: were the correct endpoints and secrets extracted? We also measure hallucination via text matching, using small local models, and with a traditional baseline at graded obfuscation levels.

3. Comparison of the Two Ideas

	Idea 1: Nmap → CVEs	**Idea 2: JavaScript code**
The model reads	Nmap output	JS code (minified / obfuscated)
Correct answer	From NVD	Endpoints and secrets I plant myself
Hallucination measure	Does the CVE exist in NVD and apply to the version?	Is the item present in the original code?
Baseline	CPE from Nmap + NVD lookup	Regex or a ready-made tool
Difficulty	Easy to medium	Easy to medium
Note	The most clearly defined and safest for the time available	Both approaches may fail under heavy obfuscation; the result is reported as a limit of the methods

4. Timeline (for the idea that will be adopted)

Period	Work
Until October 31	Prepare the environment, build the correct answer set, and read closely related papers.
November 1 – 8	Write the scripts, run the experiments, and collect the numbers.
November 9 – 14	Write the paper in the IEEE template and review it.
November 15, 2026	Final paper submission deadline for MASTER 2027 (acceptance notification December 15; conference January 27–28, 2027).

5. Ethical Considerations

All experiments are run on legal training targets and applications that I own, inside Docker on my personal machine. The secrets used are entirely fake, and no system or website that I do not own, or lack explicit permission to test, will be scanned.

Note: the prior work cited above is the result of a preliminary search; a detailed literature review (IEEE Xplore / Google Scholar) is in progress to confirm that the proposed contribution has not already been made in the same form.

> I would welcome your guidance on choosing the more suitable of the two ideas, the scope of the experiment, and its fit for the conference. Thank you for your time.